

Project Proposal

Automated Classification of White Blood Cell Subtypes
Using Convolutional Neural Networks

By:

Yasmin Rocio Orduz Landazabal
Matthew Ragsdale

Submitted to:

Dr. Meenalosini Vimal Cruz

For

Advanced Computer Vision and Deep Learning
CSCI 7090

Department of Computer Science
Georgia Southern University
Fall 2026

1 Problem Statement

Differential blood counts, in which a technician visually classifies white blood cells by subtype, remain a routine but labor-intensive part of clinical diagnostics. The process is prone to fatigue-driven error, and differential counts are among the most ordered tests for detecting infection, allergic response, and blood disorders. A technician working through hundreds of cells per slide, for instance, may struggle to distinguish borderline cases such as monocytes from lymphocytes after hours of repetitive counting. This project aims to build and evaluate a convolutional neural network that classifies white blood cells into four subtypes — eosinophil, lymphocyte, monocyte, and neutrophil — from microscopic images, and to assess whether an automated model can approach the accuracy reported for prior automated classification methods on the same public dataset.

2 Current Solutions and Related Work

The clinical gold standard is still a trained human looking at a stained smear under a microscope. That review can be accurate in skilled hands, but it is slow, and the error rate rises when the same person has already counted hundreds of cells. Automated methods try to take that load off the microscope, yet the published systems that use this dataset — or close copies of it — still leave a gap between a headline score and a result we can compare against.

Praveen et al. (2021) treat white-cell typing as detection plus classification. They run YOLOv3 on the same Kaggle blood-cell collection this project uses and report 99.2% detection accuracy with 90% subtype classification. The detection number is not the number this project will be judged on: our images are already cropped to a single cell, so the task is the 90% classification figure, not box-finding. Their Faster R-CNN with VGG-16 comparison sits below YOLOv3 on both tasks, which suggests that architecture choice moves the score, but it does not tell us how a standard image classifier would do once localization is no longer part of the problem.

Transfer-learning CNNs are the other common answer. Davamani et al. (2025) fine-tune deep backbones on histopathological white-cell images for clinical-practice classification and report strong overall accuracy. Gupta and Mishra (2024) likewise use a customized transfer-learning head on white-cell images. Both papers treat success as one overall percentage. That is useful as a published ceiling, and it is a weakness for this course: a single score hides whether monocytes and lymphocytes — the pair in the opening example — are the cells the model actually confuses. Özcan et al. (2024) go further and combine classification with segmentation; the extra pipeline is more than this implementation project needs, and it still leaves the four-class accuracy question only partly answered.

Thus the existing methods are either a detector scored at 90% on this dataset, or large transfer-learning systems scored as one number on related stains. We do not propose a new architecture. We will fine-tune ResNet-50 (He et al., 2016) on the already-chosen public set and report overall accuracy against Praveen et al.’s 90% classification figure, together with per-class precision, recall, and a confusion matrix, so the comparison is specific rather than a second headline.

3 Proposed Approach and Implementation Outline

We will use ResNet-50 (He et al., 2016) with weights trained on ImageNet (Russakovsky et al., 2015) and replace the final fully connected layer with a four-way softmax over eosinophil, lymphocyte, monocyte, and neutrophil. Input images are 224×224 RGB, the size that residual networks expect. A small convolutional network trained from scratch may be run as a weak baseline so we can say

whether transfer learning moved the score; it is not a second research track. The framework is PyTorch with torchvision (Paszke et al., 2019), which we already use in this course.

Preprocessing follows the folders the dataset already ships. We load the Kaggle train and test folders shipped as dataset2-master, reserve 10% of the training folder for validation — an effective split of about 72% train / 8% validation / 20% test — and leave the official test folder untouched until the final evaluation. Each JPEG is resized from 320×240 to 224×224 , then normalized with the ImageNet mean and standard deviation. Training images get a light augmentation pass — horizontal flip, a small rotation, and mild color jitter. The collection is already heavily augmented, so we will not invent a second heavy pipeline on top of that: stacked transforms on an already-augmented set are a known way to leak copies of the same cell across splits (Shorten & Khoshgoftaar, 2019).

Training is transfer learning: cross-entropy loss, Adam (Kingma & Ba, 2015), a fixed random seed, and early stopping on validation loss. If a GPU is available we fine-tune for the full schedule; if we are on CPU we cut the epoch budget rather than change the problem. Evaluation uses only the held-out official test folder. We will report overall accuracy, per-class precision and recall, macro-F1, and a confusion matrix, and we will set those numbers next to Praveen et al.’s 90% classification result.

The actual run may differ from this outline — batch size, which layers stay frozen, or how long we train — if the data or the hardware force a change. The problem statement and the dataset will not change.

4 Dataset Description

This project uses the Blood Cell Images collection published by Mooney (2018) on Kaggle, derived from the open BCCD (Blood Cell Count and Detection) set. We verified the counts, license, and folder layout against that source rather than assuming the page summary was complete.

The four classes sit near 25% in both the training and test folders, so the model likely will not need heavy class-imbalance correction such as oversampling or a weighted loss — though we will confirm that once the files are loaded and counted in Weeks 1–2. Each image is a single, pre-cropped white blood cell on a Giemsa/Wright-stained smear. Image quality is generally consistent, though resolution and staining tone vary slightly depending on which mirrored copy is downloaded, which is common for a set that has been re-hosted for several years.

The same collection appears throughout the white-cell classification literature, which is why we can put our test accuracy next to previously reported figures instead of inventing a private baseline. At roughly 12,500 images the archive is well under 1 GB and will fit on a laptop or the university cluster.

The license is MIT, as stated in the Acknowledgements on the Kaggle page, which credits the original BCCD_Dataset repository. That permits free use for academic and research work. One inconsistency on Kaggle itself has to be named: the page states the size as 12,500 images in one place and 2,500 in another, almost certainly a typo, because the file browser shows roughly 13,200 total files — a figure consistent with $\sim 12,500$ images plus label and metadata files, not 2,500. This proposal uses the 12,500 figure, a choice supported by Praveen et al. (2021), who worked with this same dataset and documented a nearly identical total of 12,396 images.

The native files are 320×240 RGB JPEGs. ResNet-50 was trained at 224×224 on ImageNet (He et al., 2016; Russakovsky et al., 2015), so every loader will resize with bilinear interpolation and then apply the ImageNet mean and standard deviation. That resize is a loss of a few pixels on the long side; we accept it so the pretrained filters stay on the grid they were trained for, rather

than stretching the residual blocks to a custom input size. Class folders on disk are expected to be named for the four subtypes already listed; we will print a per-folder count in Week 2 before any training starts.

The bundled 410-image raw set with bounding boxes is a detection corpus, not a four-class classification corpus, so we will not mix it into the train or test loaders. If a later week turns up a missing class folder or a count that does not match Table 1, we will record the loaded numbers and keep the official test tree sealed rather than rebuild a split by hand.

Table 1 records the facts we will treat as fixed. As we can see in Table 1, the set is specific, public, licensed for class use, and large enough that a four-class CNN is a reasonable fit rather than a guess.

Table 1—Blood Cell Images dataset facts used in this proposal.

Field	Value
Dataset name	Blood Cell Images
Source	Kaggle (paultimothymooney/blood-cells), derived from BCCD
URL	kaggle.com/datasets/paultimothymooney/blood-cells
Images / samples	~12,500 augmented JPEGs (~3,000 per class). Praveen et al. (2021) document 9,966 train + 2,430 test = 12,396. A smaller raw set (410 boxed images) is for detection and is not used.
Classes	4 — eosinophil, lymphocyte, monocyte, neutrophil
Train distribution	Eosinophil 2,497; lymphocyte 2,483; monocyte 2,487; neutrophil 2,499 (~25% each)
Test distribution	Eosinophil 574; lymphocyte 620; monocyte 620; neutrophil 616 (~25% each)
Dimensions and format	JPEG, RGB, 320 × 240 px (Praveen et al., 2021)
Train / val / test	Ships ~80/20 train/test. We reserve 10% of train for validation (~72% / 8% / 20%).
License	MIT, as stated on Kaggle (crediting BCCD_Dataset)

5 Expected Output and Evaluation Metrics

Success, in plain terms, is a program that takes one blood-cell photograph and names the white-cell type, together with a short report of how often that name is right on images the model has not trained on. We are not building a clinical deployment. The final report will show the test accuracy, a per-class precision/recall table, and a confusion-matrix figure.

The metrics are overall accuracy on the official test folder; per-class precision and recall; and macro-F1, so a strong majority class cannot hide a weak pair. The confusion matrix is how we will see whether monocytes and lymphocytes — the example in Section 1 — are in fact the cells the model mixes up.

The published baseline we will stand next to is Praveen et al. (2021): 90% subtype classification on this dataset. Other transfer-learning CNNs on related white-cell stains often sit in the low-to-mid 90s (Davamani et al., 2025; Gupta & Mishra, 2024). We aim for at least 90% test accuracy, and for the low 90s if fine-tuning is stable. That target is a match to published work on this task, not a claim that the model replaces a technician.

6 Constraints and Feasibility

The semester is ten working weeks after this proposal. Week 1 is dataset inspection and this document. Weeks 2–3 are download, a count of the real folder sizes, the 10% validation hold-out, and the PyTorch loaders. Weeks 4–6 are the ResNet-50 baseline, fine-tuning, and a short hyperparameter pass. Weeks 7–8 are the held-out test run, the confusion matrix, and a note on which classes are confused. Weeks 9–10 are the final paper and the presentation. Writing is part of that last block, not an afterthought.

A set of 12,500 images is large enough that a GPU is strongly recommended. We will train on a personal GPU if one is available, otherwise on the university cluster. Fine-tuning ResNet-50 on this collection should take under a few hours on GPU and substantially longer on CPU. If cluster time is scarce, transfer learning with a frozen backbone and fewer epochs is the fallback; we will not train a deep net from scratch on CPU. The archive itself is well under 1 GB, so storage is not the constraint.

The main risks are in the data, not the disk. The Kaggle images are already augmented, so a careless split could leak a transformed copy of a training cell into the test folder; we avoid that by keeping the official test tree sealed. Stain and resolution drift across mirrors can shift the pixel statistics we saw in the documentation. A from-scratch CNN can overfit; that is why the from-scratch net is only a weak baseline. Cluster queues can eat a week. Class imbalance is a low risk on the published counts, but we will check the loaded folders rather than assume the table is still exact.

Thus the project is one four-class CNN on one public set, with a published 90% number to beat or match, and a calendar that leaves time to write the result down.

References

- Davamani, K. A., Jawahar, M., Anbarasi, L. J., Ravi, V., Al Mazroa, A., & Robin, C. R. R. (2025). Deep transfer learning technique to detect white blood cell classification in regular clinical practice using histopathological images. *Multimedia Tools and Applications*, 84(9), 5699–5723. <https://doi.org/10.1007/s11042-024-19133-8>
- Gupta, S., & Mishra, A. (2024). Deep transfer learning-based classification of white blood cells using customized classification base. In *Proceedings of the 2024 Sixteenth International Conference on Contemporary Computing (IC3 2024)*. Association for Computing Machinery. <https://doi.org/10.1145/3675888.3676117>
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 770–778). <https://doi.org/10.1109/CVPR.2016.90>
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations*. <https://arxiv.org/abs/1412.6980>
- Mooney, P. (2018). Blood cell images [Data set]. Kaggle. <https://www.kaggle.com/datasets/paultimothymooney/blood-cells>
- Özcan, Ş. N., Uyar, T., & Karayegen, G. (2024). Comprehensive data analysis of white blood cells with classification and segmentation by using deep learning approaches. *Cytometry Part A*, 105(7), 501–520. <https://doi.org/10.1002/cyto.a.24839>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). PyTorch: An impera-

tive style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*. <https://arxiv.org/abs/1912.01703>

Praveen, N., Sonbhadra, S. K., Syafrullah, M., Pun, N. S., Agarwal, S., & Adiyarta, K. (2021). White blood cell subtype detection and classification. arXiv. <https://arxiv.org/abs/2108.04614>

Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A. C., & Fei-Fei, L. (2015). ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3), 211–252. <https://doi.org/10.1007/s11263-015-0816-y>

Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data*, 6, Article 60. <https://doi.org/10.1186/s40537-019-0197-0>